Sortowanie polifazowe

1. Wprowadzenie

Sortowanie polifazowe jest metodą sortowania zewnętrznego opartą na trójtaśmowym scalaniu serii, która minimalizuje liczbę operacji dyskowych poprzez optymalny rozkład początkowych serii. W przeciwieństwie do klasycznego sortowania naturalnego, sortowanie polifazowe wykorzystuje rozkład serii zgodny z ciągiem Fibonacciego, co pozwala znacznie przyspieszyć proces sortowania.

W ramach tego projektu dokonałem implementacji tego algorytmu i przeprowadziłem pomiary jego działania dla różnych rozmiarów danych, porównując je z wartościami teoretycznymi.

2. Opis algorytmu

2.1. Dystrybucja początkowa

W sortowaniu naturalnym podczas pierwszej dystrybucji serii na dwie taśmy, serie są zapisywane na zmianę, co sprawia, że na obu taśmach będzie w przybliżeniu taka sama liczba serii.

W sortowaniu polifazowym etap ten wygląda zupełnie inaczej, serie rozkłada się w taki sposób, aby ich liczba na pierwszej i drugiej taśmie odpowiadała dwóm sąsiednim liczbom w ciągu Fibonacciego. Przykładowo, jeżeli mamy początkowo 8 serii to dzielimy je na 5 i 3, a gdy 13 na 8 i 5 itd.

2.2. Fazy scalania

Kolejne fazy tego algorytmu również różnią się względem sortowania naturalnego. W tym drugim w każdej fazie scalamy wszystkie runy z taśmy 1 z taśmą 2 do taśmy 3 i potem ponownie rozkładamy serie na taśmy, aż zostanie tylko jedna.

W sortowaniu polifazowym nie wszystkie serie biorą udział w każdej fazie. W każdej z nich scalamy tyle runów, ile jest na krótszej z dwóch aktywnych taśm. W wyniku tego, jeżeli na taśmach było x i ϕx serii to po pierwszej fazie otrzymamy taśmę o x seriach a taśmy, z których scalaliśmy, będą miały ich po 0 i $(\phi - 1)x$.

Stosunek liczby serii na tych taśmach wynosi wtedy:

$$\frac{x}{(\phi - 1)x} = \frac{1}{\phi - 1} = \phi$$

Oznacza to, że po każdej fazie liczby serii ponownie tworzą parę sąsiednich wyrazów ciągu Fibonacciego. W konsekwencji, jeżeli początkowo mieliśmy F_n i F_{n-1} serii na taśmach to teraz mamy F_{n-1} i F_{n-2}. Proces ten powtarza się aż do momentu, gdy na taśmach zostaną liczby {1, 0}, czyli plik zostanie posortowany.

Warto również zauważyć, że w trakcie działania algorytmu każda taśma co trzy fazy zmienia swoją rolę – raz jest źródłem danych, a raz taśmą docelową. Jest to kolejna cecha, która różni ten algorytm od sortowania naturalnego, gdzie taśmy mają stałe role.

2.3. Problemy z początkową dystrybucją serii

W praktyce często zdarza się, że początkowa liczba serii nie jest sumą dwóch sąsiednich liczb Fibonacciego. Na szczęście jest na to bardzo proste rozwiązanie, które rozwiązuje problem od razu w pierwszej fazie algorytmu.

W przypadku, jeżeli mamy liczbę np. 12, która nie jest elementem ciągu Fibonacciego, bierzemy największą mniejszą liczbę od 12, która jest w ciągu Fibonacciego, w tym przypadku jest to 8. Możemy zapisać 12 jako 8+4, podczas gdy idealny rozkład to 8+5.

Brakująca seria nie stanowi jednak problemu, ponieważ w pierwszej fazie można „udawać", że istnieje dodatkowa seria (dummy). Podczas scalania nie będziemy scalać 4 serii z obu taśm, lecz 5 i skorzystamy z faktu, że podczas scalania x serii z y seriami otrzymujemy $max(x, y)$ serii. Oznacza to, że jeżeli scalimy 5 serii z 4 seriami, to otrzymamy 5 serii.

W takiej sytuacji jak nasza dystrybucja na taśmach wynosiła 8 4 0, to po pierwszej fazie będzie 3 0 5 i już mamy liczby, które się zgadzają z ciągiem Fibonacciego.

3. Liczba operacji algorytmu

Oznaczenia:

- N – liczba rekordów w pliku,
- r – początkowa liczba serii,
- b – współczynnik blokowania (liczba rekordów w jednym bloku),
- ϕ – złota liczba $\left(\frac{1+\sqrt{5}}{2}\right)$,
- k – liczba faz,
- α – udział rekordów biorących udział w scalaniu w pojedynczej fazie.

3.1. Liczba faz

Po każdej fazie liczba serii maleje z F_n do F_{n-1}. Ponieważ proces kończy się, kiedy osiągniemy liczbę serii równą 1, liczba serii jest równa w przybliżeniu n.

Korzystając ze wzoru Bineta:

$$F_n = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n - \frac{1}{\sqrt{5}}\left(\frac{1-\sqrt{5}}{2}\right)^n$$

Ponieważ drugi człon tego wyrażenia szybko zbiera do zera, można przybliżyć go do:

$$F_n = \frac{1}{\sqrt{5}}\left(\frac{1+\sqrt{5}}{2}\right)^n = \frac{\phi^n}{\sqrt{5}}$$

Skoro w naszym przypadku $r = F_n$, otrzymujemy:

$$r = \frac{\phi^k}{\sqrt{5}}$$
$$\phi^k = \sqrt{5}r$$
$$k = \log_\phi \sqrt{5}r$$
$$k = \log_\phi \sqrt{5} + \log_\phi r$$

W obliczeniach pomijamy stały składnik $\log_\phi \sqrt{5}$, gdyż jest to stała o małej wartości (1.67) i usunięcie jej ułatwi późniejsze obliczenia.

$$k = \log_\phi r$$
$$k = \frac{\log_2 r}{\log_2 \phi}$$
$$k = \frac{\log_2 r}{0.69} = \mathbf{1.44 \log_2 r}$$

Jeżeli jednak zależy nam na jeszcze dokładniejszym ustaleniu ilości faz, to możemy nie odrzucać wcześniej wspomnianej stałej i wtedy otrzymamy:

$$k = \lceil \mathbf{1.44 \log_2 r + 1.67} \rceil - \mathbf{2}$$

Odjęcie 2 wynika z różnic w indeksowaniu ciągu.

3.2. Udział rekordów w scalaniu

Ciekawą cechą tego algorytmu jest fakt, że nie wszystkie rekordy biorą udział w każdej fazie. Aby obliczyć liczbę operacji algorytmu trzeba oszacować, jak wiele rekordów bierze udział w każdej z faz.

Załóżmy, że na taśmie z mniejszą ilością serii mamy x serii oraz każda seria ma średnio y rekordów. W takiej sytuacji taśma z większą ilością elementów ma około ϕx serii oraz każda seria ma około ϕy rekordów.

Oznacza to, że ilość wszystkich rekordów możemy opisać jako:

$$xy + \phi^2 xy = xy(1 + \phi^2)$$

W każdej fazie scalamy tyle serii, aby opróżnić krótszą taśmę, czyli x serii. Zatem ilość wszystkich rekordów biorących udział w scalaniu możemy opisać jako:

$$xy + \phi xy = xy(1 + \phi)$$

Iloraz tych dwóch wartości jest naszym współczynnikiem udziału rekordów:

$$\alpha = \frac{xy(1+\phi)}{xy(1+\phi^2)} = \frac{1+\phi}{1+\phi^2} = \frac{2.62}{3.62} = \mathbf{0.72}$$

3.3. Liczba operacji dyskowych

Gdyby w każdej fazie brały udział wszystkie rekordy, to wzór na liczbę wszystkich operacji podczas scalania wyglądałby tak: $\frac{2N}{b}k$. Dzielimy przez b, ponieważ przetwarzamy dane blokowo i przy każdej operacji odczytujemy od razu b rekordów. Dodatkowo 2 z przodu jest nam potrzebna, ponieważ w każdej fazie każdy rekord w niej biorący udział jest odczytywany i zapisywany dokładnie raz.

Jednak w naszym przypadku faza nie bierze pod uwagę wszystkich rekordów, trzeba więc zmodyfikować ten wzór o współczynnik α:

$$\frac{2N}{b}\alpha k$$

Należy jednak pamiętać również o pierwszej części algorytmu, która polega na dystrybucji początkowej serii na dwie taśmy, potrzebujemy na to $\frac{2N}{b}$ operacji, więc nasz wzór ostatecznie to:

$$\frac{2N}{b}(\alpha k + 1) = \boldsymbol{\frac{2N}{b}(1.04\log_2 r + 1)}$$

Jeżeli ponownie chcemy uzyskać dokładniejsze przybliżenie ilości operacji, możemy ten wzór podstawić jako:

$$\boldsymbol{2\left\lceil\frac{N}{b}\right\rceil(\alpha k + 1)}$$

$$\boldsymbol{2\left\lceil\frac{N}{b}\right\rceil(0.72 * (\lceil 1.44\log_2 r + 1.67\rceil - 2) + 1)}$$

4. Porównanie implementacji z teorią

Zaimplementowany przeze mnie program przyjmuje pliki tekstowe, w których każdy rekord znajduje się w osobnej linii, a pomiędzy każdym z pól rekordu jest spacja. W przypadku mojego typu rekordu, którym jest wycinek koła - kąt i promień, są to dwie liczby zmiennoprzecinkowe typu double oddzielone spacją (16 bajtów w sumie). Program zlicza liczbę operacji dyskowych (osobno odczytów i zapisów) wykonywanych podczas trwania algorytmu oraz liczy, ile potrzebnych było faz scalania.

W mojej implementacji dodałem dodatkowy etap, w którym zliczam początkową ilość serii w pliku, ponieważ nie udało mi się wymyślić jak połączyć ten etap z początkową dystrybucją, więc moja implementacja potrzebuje $\left\lceil\frac{N}{b}\right\rceil$ więcej operacji dyskowych niż wcześniej obliczona wartość teoretyczna. Biorąc to pod uwagę, można by zmodyfikować nasz wzór:

$$2\left\lceil\frac{N}{b}\right\rceil(\alpha k + \mathbf{1.5})$$

Eksperyment polegał na uruchomieniu programu dla plików o liczbie rekordów od 64 do około 1 miliona. Podczas pomiarów jako **współczynnik blokowania** wykorzystywałem liczbę **256**. W trakcie testów używałem losowo wygenerowane dane, co sprawia, że możemy **ilość serii r** opisać w przybliżeniu jako wartość $\boldsymbol{\frac{N}{2}}$**.**

4.1. Ilość faz

N	Ilość faz	Ilość faz (teor.)
64	7	7
128	9	9
256	10	10
512	12	12
1024	13	13
2048	15	15
4096	16	16
8192	17	17
16384	19	19
32768	20	20
65536	22	22
131072	23	23
262144	25	25
524288	26	26
1048576	28	28

Pomierzona liczba faz jest identyczna z wartością teoretyczną, co potwierdza poprawność implementacji i zgodność z modelem.

4.2. Ilość operacji dyskowych

Porównanie pomierzonych operacji dyskowych z wzorem teoretycznym (bez brania pod uwagę dodatkowego etapu):

N	R	W	R+W	R+W (teor.)	Błąd względny
64	10	9	19	12,13	0,566
128	12	11	23	15,02	0,531
256	15	12	27	16,47	0,639
512	31	27	58	38,73	0,497
1 024	53	47	100	83,26	0,201
2 048	112	102	214	189,67	0,128
4 096	231	213	444	402,49	0,103
8 192	480	446	926	851,28	0,088
16 384	1 040	974	2 014	1 887,81	0,067
32 768	2 158	2 028	4 186	3 960,87	0,057
65 536	4 666	4 408	9 074	8 662,71	0,047
131 072	9 714	9 200	18 914	18 066,39	0,047
262 144	20 830	19 804	40 634	39 096,67	0,039
524 288	43 258	41 208	84 466	81 157,23	0,041
1 048 576	91 833	87 785	179 618	174 170,03	0,031

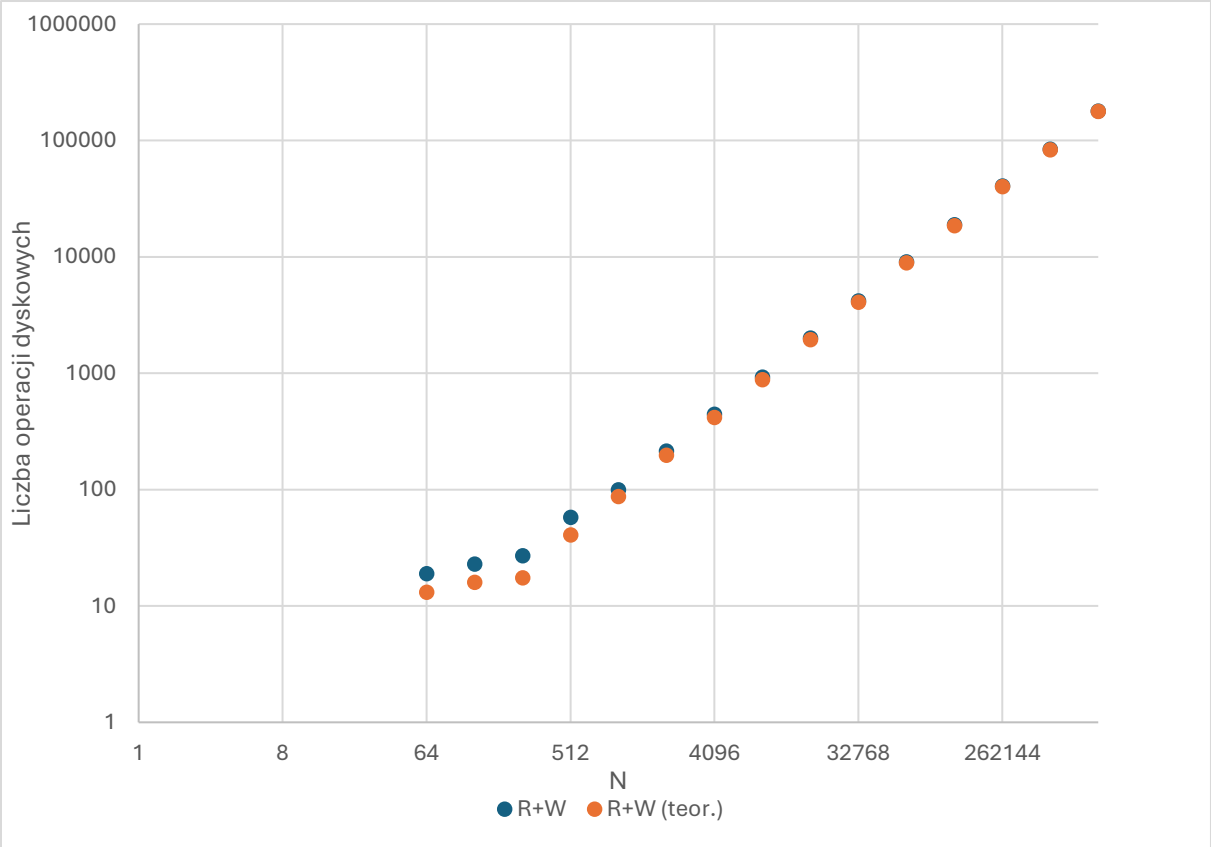
Tutaj łatwo zauważyć, że wyniki trochę się różnią od wartości teoretycznych, ale widać też, że na ogół im większe dane bierzemy, tym błąd jest coraz mniejszy. Kolejna obserwacja, jaką można zrobić to fakt, że liczba odczytów jest większa niż liczba zapisów i wynika to z dwóch przyczyn:

- W przypadku, gdy liczba rekordów jest podzielna przez b, program wykonuje dodatkowy odczyt w celu potwierdzenia końca pliku,
- Dodatkowy etap zliczania runów wymaga $\frac{N}{b}$ dodatkowych odczytów.

Porównajmy teraz pomiary ze wzorem teoretycznym, który bierze pod uwagę początkowy etap przeze mnie zaimplementowany:

N	R+W	R+W (teor.)	Błąd względny
64	19	13,13	0,447
128	23	16,02	0,435
256	27	17,47	0,545
512	58	40,73	0,424
1 024	100	87,26	0,146
2 048	214	197,67	0,083
4 096	444	418,49	0,061
8 192	926	883,28	0,048
16 384	2 014	1 951,81	0,032
32 768	4 186	4 088,87	0,024
65 536	9 074	8 918,71	0,017
131 072	18 914	18 578,39	0,018
262 144	40 634	40 120,67	0,013
524 288	84 466	83 205,23	0,015
1 048 576	179 618	178 266,03	0,008

Porównując wyniki z rozszerzonym wzorem teoretycznym, otrzymano bardzo dobrą zgodność - dla największych danych (ponad 1 mln rekordów) błąd wynosi poniżej 1%.



Na wykresie przedstawiono zależność liczby operacji dyskowych (R+W) od liczby rekordów N. Można zauważyć, że trend wzrostu liczby operacji dyskowych w moich pomiarach jest taki sam jak w wyliczeniach teoretycznych.

Warto zauważyć, że pierwsze trzy punkty pomiarowe wykazują nieco inny wzrost liczby operacji niż pozostałe. Wynika to z faktu, że w tych testach zachodziło $N \leq b$, co oznacza, że do odczytania całego pliku potrzebna była tylko jedna operacja. W kolejnych testach, gdy $N > b$, wartość $\left\lceil \frac{N}{b} \right\rceil$ rośnie proporcjonalnie do wzrostu danych. W związku z tym, że w moim eksperymencie każdy kolejny plik miał dwa razy więcej rekordów niż poprzedni, to wartość ta się podwajała.

5. Podsumowanie

Przeprowadzone testy potwierdzają teoretyczne właściwości sortowania polifazowego. Liczba faz oraz liczba operacji dyskowych odpowiadają wartością oczekiwanym, a różnice znikały wraz ze wzrostem ilości danych. Obserwowany niewielki błąd wynika z faktu okazjonalnych dodatkowych odczytów na końcu pliku oraz faktu, że ten model teoretyczny jest jedynie przybliżeniem. Wyniki eksperymentu wskazują na to, że implementacja działa poprawnie i jest zgodna z modelem teoretycznym.